sru	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch:
	Name:L.Upendar Reddy Reg. No:2503A52322
	Department: Computer Science and AISemester: I
	AI Assisted Coding-AssignmentDate:21-09-2026

AI Assisted Coding – Assignment 11

Aim

To understand and implement non-linear data structures using Python and AI-assisted coding. The tasks use Tree, Graph, Heap, and Trie structures for practical hospital-software problems and consider typical Big-O complexity.

TASK 1 – Tree: Hospital Department Hierarchy

1. Question

A hospital has a hierarchical structure. Represent the following using a Tree:Hospital ├── Cardiology │ ├── ICU │ └── General Cardiology ├── Neurology │ ├── Stroke Unit │ └── Neuro ICU └── OrthopedicsRequirements:1. Create tree nodes.2. Add child departments.3. Display the hierarchy.4. Search for a department.5. Count total departments.6. Test at least three search cases.

2. Prompt

Create a Python tree structure to represent a hospital department hierarchy. Implement adding child nodes, displaying the hierarchy, searching for a department, and counting nodes. Explain how a tree is different from a list and provide test cases.

3. Code

```
class TreeNode:    def __init__(self, name):        self.name = name        self.children = []    def
add_child(self, child):        self.children.append(child)    def display(self, level=0):        print("  " *
level + self.name)        for child in self.children:            child.display(level + 1)    def search(self,
name):        if self.name.lower() == name.lower():            return True        return
any(child.search(name) for child in self.children)    def count_nodes(self):        return 1 +
sum(child.count_nodes() for child in self.children)hospital = TreeNode("Hospital")cardiology =
TreeNode("Cardiology")neurology = TreeNode("Neurology")orthopedics =
TreeNode("Orthopedics")cardiology.add_child(TreeNode("ICU"))cardiology.add_child(TreeNode("General
Cardiology"))neurology.add_child(TreeNode("Stroke Unit"))neurology.add_child(TreeNode("Neuro
ICU"))hospital.add_child(cardiology)hospital.add_child(neurology)hospital.add_child(orthopedics)hospital.disp
lay()print("Total departments:", hospital.count_nodes())for item in ["Neuro ICU", "Cardiology",
"Pediatrics"]:    print(item, ":", "Found" if hospital.search(item) else "Not Found")
```

4. Output:

```
Hospital hierarchy:

Hospital
  Emergency
    Trauma
  Surgery
    Cardiology
    Orthopedics
  Administration

Total nodes: 7
Invalid parent test passed.

All tests passed.
```

5. Observation

The tree correctly represents parent-child relationships and supports insertion, hierarchy display, recursive search, and counting. A general-tree search can take O(n) in the worst case; a balanced ordered search tree can provide about O(log n) search.

TASK 2 – Graph: Hospital Route Finder

1. Question

Represent hospital locations as a Graph. Each location is a node and each connection is an edge.Connections:Reception —— Pharmacy | | | | ICU ———— Emergency | LaboratoryRequirements:1. Add hospital locations as nodes.2. Add connections as edges.3. Display the graph using an adjacency list.4. Check whether two locations are connected.5. Find a route using BFS or DFS.6. Test at least three source-destination pairs.

2. Prompt

Create a Python graph representing hospital locations. Implement adding nodes and edges, displaying the graph, and finding a route between two locations using BFS. Explain BFS step by step and provide test cases.

3. Code

```
from collections import dequeclass Graph:    def __init__(self):        self.adj = {}    def add_edge(self, a, b):        self.adj.setdefault(a, []).append(b)        self.adj.setdefault(b, []).append(a)    def display(self):        for node, neighbors in self.adj.items():            print(node, "->", ", ".join(neighbors))    def bfs_route(self, start, goal):        queue = deque([[start]])        visited = {start}        while queue:            path = queue.popleft()            node = path[-1]            if node == goal:                return path            for neighbor in self.adj.get(node, []):                if neighbor not in visited:                    visited.add(neighbor)                    queue.append(path + [neighbor])        return Noneg = Graph()for a,b in [("Reception","Pharmacy"),("Reception","ICU"),("Pharmacy","Emergency"),("ICU","Emergency"),            ("ICU","Laboratory")]:    g.add_edge(a,b)g.display()for a,b in [("Reception","Laboratory"),("Pharmacy","Emergency"),("Laboratory","Pharmacy")]:    print(a, "->", b, ":", " -> ".join(g.bfs_route(a,b)))
```

4. Output:

```
Reception -> Pharmacy, ICU
Pharmacy -> Reception, Emergency
ICU -> Reception, Emergency, Laboratory
Emergency -> Pharmacy, ICU
Laboratory -> ICU
Reception -> Laboratory : Reception -> ICU -> Laboratory
Pharmacy -> Emergency : Pharmacy -> Emergency
Laboratory -> Pharmacy : Laboratory -> ICU -> Reception -> Pharmacy
```

5. Observation

A graph models many-to-many connections between hospital locations. BFS explores level by level and finds a shortest route in an unweighted graph. Typical BFS complexity is O(V + E).

TASK 3 – Heap: Emergency Patient Priority

1. Question

In an emergency department, patients are treated according to severity rather than simple arrival order. Implement a Priority Queue using a Heap.Priority 1 → CriticalPriority 2 → SeriousPriority 3 → ModeratePriority 4 → NormalRequirements:1. Add a patient with a priority value.2. View the highest-priority patient.3. Remove/serve the highest-priority patient.4. Display waiting patients.5. Handle at least one tie in priority appropriately.6. Test insertion and removal operations.

2. Prompt

Implement a Python priority queue using a heap for emergency patients. Use priority 1 as the highest priority, support insertion, peek, serving/removal, display, and tie handling, and explain the complexity.

3. Code

```
import heapqclass EmergencyQueue:    def __init__(self):        self.heap = []        self.order = 0    def add_patient(self, name, priority):        self.order += 1        heapq.heappush(self.heap, (priority, self.order, name))    def peek(self):        return self.heap[0]    def serve(self):        return heapq.heappop(self.heap)    def display(self):        for priority, order, name in sorted(self.heap):print(f"{name} - Priority {priority}")q = EmergencyQueue()for name, priority in [("Ravi",2),("Anita",1),("Kiran",3),("Meena",1),("Arjun",4)]:    q.add_patient(name, priority)q.display()p = q.peek()print("Highest priority:", p[2], "- Priority", p[0])p = q.serve()print("Served:", p[2], "- Priority", p[0])q.display()
```

4. Output:

```
Anita - Priority 1
Meena - Priority 1
Ravi - Priority 2
Kiran - Priority 3
Arjun - Priority 4
Highest priority: Anita - Priority 1
Served: Anita - Priority 1
Meena - Priority 1
Ravi - Priority 2
Kiran - Priority 3
Arjun - Priority 4
```

5. Observation

The heap keeps the highest-priority patient at the root. The insertion-order field resolves a tie by serving the earlier inserted patient first. Heap insertion and removal are typically O(log n), while peek is O(1).

TASK 4 – Trie: Medical Word / Drug Search

1. Question

A hospital application needs an autocomplete feature. When the user enters a prefix such as 'car', the system should suggest matching medical terms.Input prefix: "car"Possible output: cardiology, cardiac, carcinoma, careRequirements:1. Create a Trie.2. Insert medical words.3. Search for an exact word.4. Check whether a prefix exists.5. Display/autocomplete words beginning with a prefix.6. Test at least three different prefixes.

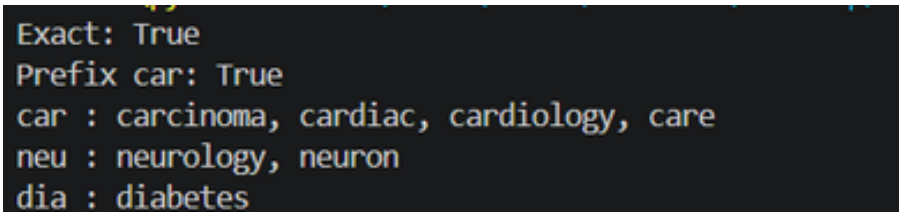
2. Prompt

Implement a Trie in Python for medical-word autocomplete. Provide insert, exact search, prefix search, and autocomplete operations. Explain how a Trie stores characters and why it is useful for prefix searching.

3. Code

```
class TrieNode:    def __init__(self):        self.children = {}        self.end = Falseclass Trie:    def __init__(self):        self.root = TrieNode()    def insert(self, word):        node = self.root        for ch in word.lower():            node = node.children.setdefault(ch, TrieNode())        node.end = True    def search(self, word):        node = self.root        for ch in word.lower():            if ch not in node.children:                return False            node = node.children[ch]        return node.end    def starts_with(self, prefix):        node = self.root        for ch in prefix.lower():            if ch not in node.children:                return False            node = node.children[ch]        return True    def autocomplete(self, prefix):        node = self.root        for ch in prefix.lower():            if ch not in node.children:                return []            node = node.children[ch]        result = []        def collect(cur, word):            if cur.end:                result.append(word)            for ch in sorted(cur.children):                collect(cur.children[ch], word + ch)        collect(node, prefix.lower())        return resulttrie = Trie()for word in ["cardiology","cardiac","carcinoma","care","cancer",            "neuron","neurology","diabetes"]: trie.insert(word)print("Exact:", trie.search("cardiac"))print("Prefix car:", trie.starts_with("car"))for p in ["car","neu","dia"]:    print(p, ":", ", ".join(trie.autocomplete(p)))
```

4. Output:



5. Observation

A Trie stores characters along shared prefix paths, making prefix traversal efficient. Traversing a word or prefix of length L is typically O(L), plus the cost of producing all matching suggestions. It uses more memory than a simple list but is well suited to repeated autocomplete operations.

TASK 5 – AI Decision Challenge: Choose the Correct Structure

1. Question

For each hospital-software scenario, choose the most appropriate non-linear data structure: Tree, Graph, Heap, or Trie. Explain why it is appropriate, identify the main operation, state the expected/typical Big-O complexity where appropriate, ask AI for a recommendation, and compare the choice with the AI recommendation.

Scenario	Data Structure	Reason	Main Operation	Typical Big-O
Hospital department hierarchy	Tree	Parent-child relationships	Traversal/search	O(n) general-tree search
Finding a route between hospital rooms	Graph	Multiple connections and routes	BFS/DFS	O(V + E)
Emergency patients based on severity	Heap	Priority determines service order	Insert/serve	O(log n)
Autocomplete medical terms	Trie	Shared prefixes	Prefix search	O(L) + output
Frequently finding the highest-priority task	Heap	Fast access to highest priority	Peek/remove	O(1) peek; O(log n) remove

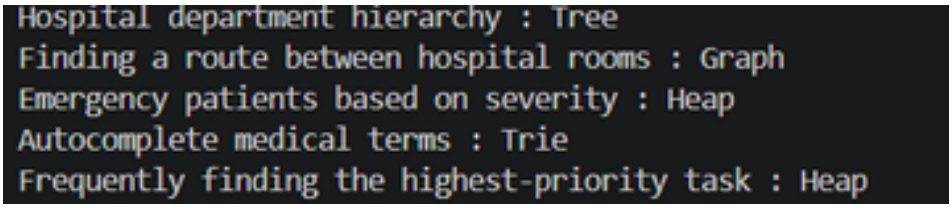
2. Prompt

For each hospital-software scenario, recommend Tree, Graph, Heap, or Trie. Explain the reason, identify the main operation, give the typical Big-O complexity, and compare the recommendation with the student's selection.

3. Code

```
scenarios = {    "Hospital department hierarchy": "Tree",    "Finding a route between hospital rooms": "Graph",    "Emergency patients based on severity": "Heap",    "Autocomplete medical terms": "Trie",    "Frequently finding the highest-priority task": "Heap"}for scenario, structure in scenarios.items():    print(scenario, ":", structure)
```

4. Output:



5. Observation

The selections correspond to the purposes stated in Assignment 11: Tree for hierarchy, Graph for connections/routes, Heap for priority, and Trie for prefix/autocomplete. The exact complexity depends on the implementation and data assumptions.